

8

COUNTING SYLLABLES FOR HAIKU POETRY



Poetry may be the supreme form of literature. It is, as Coleridge put it, “the best words in the best order.” The poet must—with great brevity—tell a story, promote an idea, describe a scene, or evoke intensity of feeling, all while obeying strict rules on rhythm and rhyme, style and structure.

Computers love rules and structure and even have the potential to evoke emotions. In his 1996 book *Virtual Muse: Experiments in Computer Poetry*, author Charles Hartman describes early attempts to write algorithms that could mimic human poetry. To quote Hartman, “The complexity of poetic interaction, the tricky dance among poet and text and reader, causes a game of hesitation. In this game, a properly programmed computer has a chance to slip in some interesting moves.”

The early programs Hartman describes could, at best, produce bad beatnik poetry. The goal at the time was to “introduce calculated bits of mechanized anarchy into the language, put the results back into the contingent world where language lives, and see how the dust settles.” As we have touched on in several chapters, context is the weak link in programming things like proper-name anagrams and null ciphers. To write computer poems that pass the “supreme” test of literature, you cannot ignore context.

Getting a computer to simulate this most human of human endeavors is an intriguing challenge—and certainly not one we can pass up. In this chapter and the next, you'll teach your computer how to generate a traditional form of Japanese poetry called *haiku*.

Japanese Haiku

Haiku consist of three lines of five, seven, and five syllables, respectively. The poems rarely rhyme, and the subject matter usually addresses the natural world—mainly the seasons—either directly or indirectly. If done properly, a haiku can immerse you in the scene, as if evoking a memory.

I've provided three example haiku here. The first is by the master Buson (1715–1783), the second by the master Issa (1763–1828), and the third by yours truly, based on memories of childhood road trips.

Standing still at dusk

Listen . . . in far distances

The song of froglings!

—*Buson*

Good friend grasshopper

Will you play the caretaker

For my little grave?

—*Issa*

Faraway cloudbanks

That I let myself pretend

Are distant mountains

—*Vaughan*

Because of its evocative nature, every haiku has a built-in “exploitable gap” for the programmer. This is summed up nicely by Peter Beilenson in his 1955 book *Japanese Haiku*: “the haiku is not expected to be always a complete or even a clear statement. The reader is supposed to add to the words his own associations and imagery, and thus to become a co-creator of his own pleasure in the poem.” Hartman adds, “The reader’s mind works most actively on sparse materials. We draw the clearest constellations from the fewest stars. So, the nonsense factor is low for a tiny collocation of words that can be imbued with imagistic significance.” To put it simply, it’s harder to mess up a short poem. Readers always assume the poet had a point and will make one up themselves if they can’t find it.

Despite this advantage, training your computer to write poetry is no mean feat, and you’ll need two whole chapters to get it done. In this chapter, you’ll write a program that counts the number of syllables in words and phrases so that you can honor the syllabic structure of the haiku. In **Chapter 9**, you’ll use a technique called *Markov chain analysis* to capture the essence of haiku—the elusive evocative component—and transform existing poems into something new and, occasionally, arguably better.

Project #15: Counting Syllables

Counting syllables in English is difficult. The problem lies in, as Charles Hartman put it, the quirky spelling and tangled linguistic history of English. For example, a word like *aged* may be one syllable or two depending on whether it describes a man or a cheese. How can a program count syllables accurately without degenerating into an endless list of special cases?

The answer is that it can’t, at least not without a “cheat sheet.”

Fortunately, these cheat sheets exist, thanks to a branch of science known as *natural language processing (NLP)*, which deals with interactions between the precise and structured language of computers and the nuanced, frequently ambiguous “natural” language used by humans.

Example uses for NLP include machine translations, spam detection, comprehension of search engine questions, and predictive text recognition for cell phone users. The biggest impact of NLP is yet to come: the mining

of vast volumes of previously unusable, poorly structured data and engaging in seamless conversations with our computer overlords.

In this chapter, you'll use an NLP dataset to help count syllables in words or phrases. You'll also write code that finds words that are missing from this dataset and then helps you build a supporting dictionary. Finally, you'll write a program to help you check your syllable-counting code. In **Chapter 9**, you'll use this syllable-counting algorithm as a module in a program that helps you computationally produce the highest achievement in literature: poetry.

THE OBJECTIVE

Write a Python program that counts the number of syllables in an English word or phrase.

The Strategy

For you and me, counting syllables is easy. Place the back of your hand just below your chin and start talking. Every time your chin hits your hand you've spoken a syllable. Computers don't have hands or chins, but every vowel sound represents a syllable—and computers can count vowel sounds. It's not easy, however, as there isn't a simple rule for doing this. Some vowels in written language are silent, such as the *e* in *like*, and some combine to make a single sound, such as the *oo* in *moo*. Luckily, the number of words in the English language isn't infinite. Fairly exhaustive lists are available that include much of the information you need.

A *corpus* is a fancy name for a body of text. In **Chapter 9**, you'll use a *training corpus*—composed of haiku—that teaches Python how to write new haiku. In this chapter, you'll use this same corpus to extract syllable counts.

Your syllable counter should evaluate both phrases and individual words, since you will ultimately use it to count the syllables in entire *lines* in a haiku. The program will take some text as an input, count the number of

syllables in each word, and return the total syllable count. You'll also have to deal with things like punctuation, whitespace, and missing words.

The primary steps you need to follow are:

1. Download a large corpus with syllable-count information.
2. Compare the syllable-count corpus to the haiku-training corpus and identify all the words missing from the syllable-count corpus.
3. Build a dictionary of the missing words and their syllable counts.
4. Write a program that uses both the syllable-count corpus and the missing-words dictionary to count syllables in the training corpus.
5. Write a program that checks the syllable-counting program against updates of the training corpus.

Using a Corpus

The Natural Language Toolkit (NLTK) is a popular suite of programs and libraries for working with human language data in Python. It was created in 2001 as part of a computational linguistics course in the Department of Computer and Information Science at the University of Pennsylvania. Development and expansion have continued with the help of dozens of contributors. To learn more, check out the official NLTK website at <http://www.nltk.org/>.

For this project, you will use NLTK to access the Carnegie Mellon University Pronouncing Dictionary (CMUdict). This corpus contains almost 125,000 words mapped to their pronunciations. It is machine readable and useful for tasks such as speech recognition.

Installing NLTK

You can find instructions for installing NLTK on Unix, Windows, and macOS at <http://www.nltk.org/install.html>. If you are using Windows, I suggest you start by opening Windows Command Prompt or PowerShell and trying to install with `pip`:

```
python -m pip install nltk
```

You can check the installation by opening the Python interactive shell and typing:

```
>>> import nltk  
>>>
```

If you don't get an error, you're good to go. Otherwise, follow the instructions on the website just cited.

Downloading CMUdict

To get access to CMUdict (or any of the other NLTK corpora), you have to download it. You can do this using the handy NLTK Downloader. Once you've installed NLTK, enter the following into the Python shell:

```
>>> import nltk  
>>> nltk.download()
```

The NLTK Downloader window (**Figure 8-1**) should now be open. Click the **Corpora** tab near the top, then click **cmudict** in the Identifier column. Next, scroll to the bottom of the window and set the Download Directory; I used the default, `C:\nltk_data`. Finally, click the **Download** button to load CMUdict.

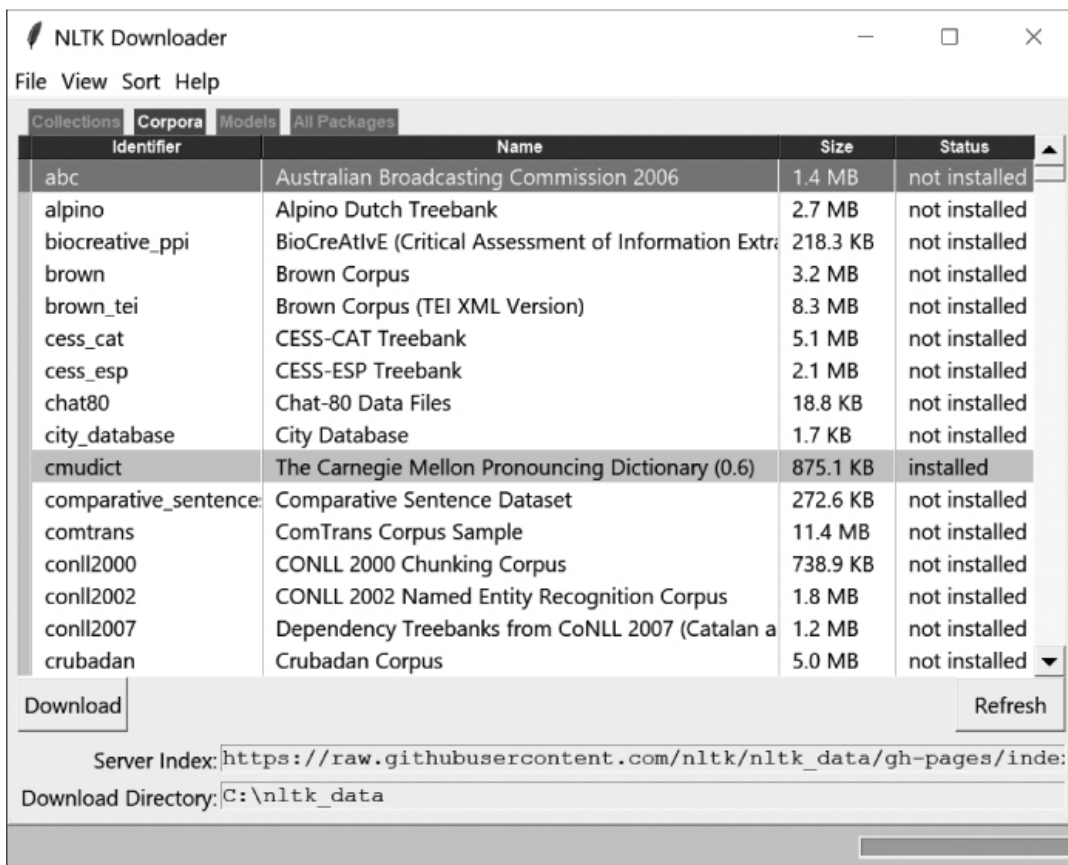


Figure 8-1: The NLTK Downloader window with *cmudict* selected for download

When CMUdict has finished downloading, exit the Downloader and enter the following into the Python interactive shell:

```
>>> from nltk.corpus import cmudict
>>>
```

If you don't encounter an error, then the corpus has been successfully downloaded.

Counting Sounds Instead of Syllables

The CMUdict corpus breaks words into sets of *phonemes*—perceptually distinct *units* of sound in a specified language—and marks vowels for lexical stress using numbers (0, 1, and 2). The CMUdict corpus marks every vowel with one, and only one, of these numbers, so you can use the numbers to identify the vowels in a word.

Looking at words as a set of phonemes will help you sidestep a few problems. For one, CMUdict will not include vowels in the written word that are unpronounced. For example, here's how CMUdict sees the word *scarecrow*:

```
[['S', 'K', 'AE1', 'R', 'K', 'R', 'OW0']]
```

Each item with a numerical suffix represents a pronounced vowel. Note that the silent *e* at the end of *scare* is correctly omitted.

Second, sometimes multiple and consecutive written vowels are pronounced as just a single phoneme. For example, this is how CMUdict represents *house*:

```
[['HH', 'AW1', 'S']]
```

Note how the corpus treats the written double vowels *ou* as a single vowel, 'AW1', for pronunciation purposes.

Handling Words with Multiple Pronunciations

As I mention in the introduction, some words have multiple distinct pronunciations; *aged* and *learned* are just two examples:

```
[['EY1', 'JH', 'D'], ['EY1', 'JH', 'IH0', 'D']]  
[['L', 'ER1', 'N', 'D'], ['L', 'ER1', 'N', 'IH0', 'D']]
```

Note the nested lists. The corpus recognizes that both words can be pronounced with one or two syllables. This means it will return more than one syllable count for certain words, something you will have to account for in your code.

Managing Missing Words

CMUdict is very useful, but with a corpus, a word is either there or not. It took only seconds to find more than 50 words—like *dewdrop*, *bathwater*,

dusky, ridgeline, storks, dragonfly, beggar, and archways—missing from CMUdict in a 1,500-word test case. So, one of your strategies should be to check CMUdict for missing words and then address any omissions by making a corpus for your corpus!

The Training Corpus

In **Chapter 9**, you’ll use a training corpus of several hundred haiku to “teach” your program how to write new ones. But you can’t count on CMUdict to contain all the words in this corpus because some will be Japanese words, like *sake*. And as you already saw, even some common English words are missing from CMUdict.

So the first order of business is to check all the words in the training corpus for membership in CMUdict. To do this, you’ll need to download the training corpus, called *train.txt*, from

<https://www.nostarch.com/impracticalpython/>. Keep it in the same folder as all the Python programs from this chapter. The file contains slightly under 300 haiku that have been randomly duplicated around 20 times to ensure a robust training set.

Once you find words that aren’t in CMUdict, you’ll write a script to help you prepare a Python dictionary that uses words as keys and syllable counts as values; then you’ll save this dictionary to a file that can support CMUdict in the syllable-counting program.

The Missing Words Code

The code in this section will find words missing from CMUdict, help you prepare a dictionary of the words and their syllable counts, and save the dictionary to a file. You can download the code from

<https://www.nostarch.com/impracticalpython/> as *missing_words_finder.py*.

Importing Modules, Loading CMUdict, and Defining the main() Function

Listing 8-1 imports modules, loads CMUdict, and defines the `main()` function that runs the program.

```
import sys
from string import punctuation
❶ import pprint
import json
from nltk.corpus import cmudict

❷ cmudict = cmudict.dict() # Carnegie Mellon University Pronouncing Dictionary

❸ def main():
    ❹ haiku = load_haiku('train.txt')
    ❺ exceptions = cmudict_missing(haiku)
    ❻ build_dict = input("\nManually build an exceptions dictionary (y/n)? \n")
    if build_dict.lower() == 'n':
        sys.exit()
    else:
        ❼ missing_words_dict = make_exceptions_dict(exceptions)
        save_exceptions(missing_words_dict)
```

Listing 8-1: Imports modules, loads CMUdict, and defines `main()`

You start with some familiar imports and a few new ones. The `pprint` module lets you “pretty print” your dictionary of missing words in an easy-to-read format ❶. You’ll write out this same dictionary as persistent data using JavaScript Object Notation (`json`), a text-based way for computers to exchange data that works well with Python data structures; it’s part of the standard library, standardized across multiple languages, and the data is secure and human readable. Finish by importing the `CMUdict` corpus.

Next, call the `cmudict` module’s `dict()` method to turn the corpus into a dictionary with the words as keys and their phonemes as values ❷.

Define the `main()` function that will call functions to load the training corpus, find missing words in `CMUdict`, build a dictionary with the words and their syllable counts, and save the results ❸. You’ll define these functions after defining `main()`.

Call the function to load the haiku-training corpus and assign the returned set to a variable named `haiku` ❹. Then call the function that will find the missing words and return them as a set ❺. Using sets removes duplicate words that you don't need. The `cmudict_missing()` function will also display the number of missing words and some other statistics.

Now, ask the user if they want to manually build a dictionary to address the missing words and assign their input to the `build_dict` variable ❻. If they want to stop, exit the program; otherwise, call a function to build the dictionary ❼ and then another one to save the dictionary. Note that the user isn't restricted to pressing `y` if they want to continue, though that's the prompt.

Loading the Training Corpus and Finding Missing Words

Listing 8-2 loads and prepares the training corpus, compares its contents to CMUdict, and keeps track of the differences. These tasks are divided between two functions.

missing_words_finder.py, part 2

```
❶ def load_haiku(filename):
    """Open and return training corpus of haiku as a set."""
    with open(filename) as in_file:
        ❷ haiku = set(in_file.read().replace('-', ' ').split())
        ❸ return haiku

def cmudict_missing(word_set):
    """Find and return words in word set missing from cmudict."""
    ❹ exceptions = set()
    for word in word_set:
        word = word.lower().strip(punctuation)
        if word.endswith("'s") or word.endswith("'s"):
            word = word[:-2]
        ❺ if word not in cmudict:
            exceptions.add(word)
    print("\nexceptions:")
    print(*exceptions, sep='\n')
```

```

❹ print("\nNumber of unique words in haiku corpus = {}".format(len(word_set)))
    print("Number of words in corpus not in cmudict = {}".format(len(exceptions)))
    membership = (1 - (len(exceptions) / len(word_set))) * 100
❺ print("cmudict membership = {:.1f}%".format(membership, '%'))
    return exceptions

```

Listing 8-2: Defines functions to load the corpus and finds words missing from CMUdict

Define a function to read in the words from the haiku-training corpus ❶. The haiku in *train.txt* have been duplicated many times, plus the original haiku contain duplicate words, like *moon*, *mountain*, and *the*. There's no point in evaluating a word more than once, so load the words as a set to remove repeats ❷. You also need to replace hyphens with spaces. Hyphens are popular in haiku, but you need to separate the words on either side in order to check for them in CMUdict. End the function by returning the haiku set ❸.

It's now time to find missing words. Define a function, `cmudict_missing()`, that takes as an argument a sequence—in this case, the set of words returned by the `load_haiku()` function. Start an empty set called `exceptions` to hold any missing words ❹. Loop through each word in the haiku set, converting it to lowercase and stripping any leading or trailing punctuation. Note that you don't want to remove interior punctuation other than hyphens because CMUdict recognizes words like *wouldn't*. Possessive words typically aren't in a corpus, so remove the trailing 's, since this won't affect the syllable count.

NOTE

Be careful of curly apostrophes (') produced by word-processing software. These are different from the straight apostrophes (') used in simple text editors and shells and may not be recognized by CMUdict. If you add new words to either the training

or JSON files, be sure to use a straight apostrophe for contractions or possessive nouns.

If the word isn't found in CMUdict, add it to `exceptions` ❶. Print these words as a check, along with some basic information ❷, like how many unique words, how many missing words, and what percentage of the training corpus are members of CMUdict. Set the percent value precision to one decimal place ❸. End the function by returning the set of exceptions.

Building a Dictionary of Missing Words

Listing 8-3 continues the *missing_words_finder.py* code, now supplementing CMUdict by assigning syllable counts to the missing words as values in a Python dictionary. Since the number of missing words should be relatively small, the user can assign the counts manually, so write the code to help them interact with the program.

missing_words_finder.py, part 3

```
❶ def make_exceptions_dict(exceptions_set):
    """Return dictionary of words & syllable counts from a set of words."""
    ❷ missing_words = {}
    print("Input # syllables in word. Mistakes can be corrected at end. \n")
    for word in exceptions_set:
        while True:
            ❸ num_sylls = input("Enter number syllables in {}: ".format(word))
            ❹ if num_sylls.isdigit():
                break
            else:
                print("
                                Not a valid answer!", file=sys.stderr)
            ❺ missing_words[word] = int(num_sylls)
        print()
    ❻ pprint.pprint(missing_words, width=1)

    ❼ print("\nMake Changes to Dictionary Before Saving?")
    print("""
    0 - Exit & Save
```

```

1 - Add a Word or Change a Syllable Count
2 - Remove a Word
""")

❶ while True:
    choice = input("\nEnter choice: ")
    if choice == '0':
        break
    elif choice == '1':
        word = input("\nWord to add or change: ")
        missing_words[word] = int(input("Enter number syllables in {}: "
                                         .format(word)))

    elif choice == '2':
        word = input("\nEnter word to delete: ")
        ❷ missing_words.pop(word, None)

    print("\nNew words or syllable changes:")
    ❸ pprint.pprint(missing_words, width=1)

    return missing_words

```

Listing 8-3: Allows the user to manually count syllables and builds a dictionary

Start by defining a function that takes the set of exceptions returned by the `cmudict_missing()` function as an argument ❶. Immediately assign an empty dictionary to a variable named `missing_words` ❷. Let the user know that if they make a mistake, they'll have a chance to fix it later; then, use a `for` and `while` loop to go through the set of missing words and present each word to the user, asking for the number of syllables as input. The word will be the dictionary key, and the `num_sylls` variable will become its value ❸. If the input is a digit ❹, break out of the loop. Otherwise, warn the user and let the `while` loop request input again. If the input passes, add the value to the dictionary as an integer ❺.

Use `pprint` to display each key/value pair on a separate line, as a check. The `width` parameter acts as a newline argument ❻.

Give the user the opportunity to make last-minute changes to the `missing_words` dictionary before saving it as a file ❷. Use triple quotes to present the options menu, followed by a `while` loop to keep the options active until the user is ready to save ❸. The three options are exiting, which invokes the `break` command; adding a new word or changing the syllable count for an existing word, which requires the word and syllable count as input; and removing an entry, which uses the dictionary `pop()` function ❹. Adding the `None` argument to `pop()` means the program won't raise a `KeyError` if the user enters a word that's not in the dictionary.

Finish by giving the user a last look at the dictionary, in the event changes were made ❺, and then return it.

Saving the Missing Words Dictionary

Persistent data is data that is preserved after a program terminates. To make the `missing_words` dictionary available for use in the *count_syllables.py* program you'll write later in this chapter, you need to save it to a file.

Listing 8-4 does just that.

missing_words_finder.py, part 4

```
❶ def save_exceptions(missing_words):  
    """Save exceptions dictionary as json file."""  
    ❷ json_string = json.dumps(missing_words)  
    ❸ f = open('missing_words.json', 'w')  
    f.write(json_string)  
    f.close()  
    ❹ print("\nFile saved as missing_words.json")  
  
❺ if __name__ == '__main__':  
    main()
```

Listing 8-4: Saves missing-words dictionary to a file and calls `main()`

Use `json` to save the dictionary. Define a new function that takes the set of missing words as an argument ❶. Assign the `missing_words` dictionary to a

new variable named `json_string` ❷; then, open a file with a `.json` extension ❸, write the `json` variable, and close the file. Display the name of the file as a reminder to the user ❹. End with the code that lets the program be run as a module or in stand-alone mode ❺.

The `json.dumps()` method serializes the `missing_words` dictionary into a string. *Serialization* is the process of converting data into a more transmittable or storable format. For example:

```
>>> import json
>>> d = {'scarecrow': 2, 'moon': 1, 'sake': 2}
>>> json.dumps(d)
'{"sake": 2, "scarecrow": 2, "moon": 1}'
```

Note that the serialized dictionary is bound by single quotes, making it a string.

I've provided a partial output from *missing_words_finder.py* here. The list of missing words at the top and the manual syllable counts at the bottom have both been shortened for brevity.

```
--snip--
froglings
scatters
paperweights
hibiscus
cumulus
nightingales

Number of unique words in haiku corpus = 1523
Number of words in corpus not in cmudict = 58
cmudict membership = 96.2%

Manually build an exceptions dictionary (y/n)?
y
Enter number syllables in woodcutter: 3
Enter number syllables in morningglory: 4
```

```
Enter number syllables in cumulus: 3
```

```
--snip--
```

Don't worry—you won't have to assign all the syllable counts. The `missing_words.json` file is complete and ready for download when you need it.

NOTE

For words that have multiple pronunciations, like jagged or our, you can force the program to use the one you prefer by manually opening the `missing_words.json` file and adding the key/value pair at any location. I did this with the word `sake` so that it uses the two-syllable Japanese pronunciation. Because word membership is checked in this file first, it will override the CMUdict value.

Now that you've addressed the holes in CMUdict, you're ready to write the code that counts syllables. In **Chapter 9**, you'll use this code as a module in the `markov_haiku.py` program.

The Count Syllables Code

This section contains the code for the `count_syllables.py` program. You'll also need the `missing_words.json` file you created in the previous section. You can download both from <https://www.nostarch.com/impracticalpython/>. Keep them together in the same folder.

Prepping, Loading, and Counting

Listing 8-5 imports the necessary modules, loads the CMUdict and missing-words dictionaries, and defines a function that will count the syllables in a given word or phrase.

`count_syllables.py`, part 1

```

import sys
from string import punctuation
import json
from nltk.corpus import cmudict

# load dictionary of words in haiku corpus but not in cmudict
with open('missing_words.json') as f:
    missing_words = json.load(f)

❶ cmudict = cmudict.dict()

❷ def count_syllables(words):
    """Use corpora to count syllables in English word or phrase."""
    # prep words for cmudict corpus
    words = words.replace('-', ' ')
    words = words.lower().split()
    ❸ num_sylls = 0
    ❹ for word in words:
        word = word.strip(punctuation)
        if word.endswith("'s") or word.endswith("'s"):
            word = word[:-2]
        ❺ if word in missing_words:
            num_sylls += missing_words[word]
        else:
            ❻ for phonemes in cmudict[word][0]:
                for phoneme in phonemes:
                    ❼ if phoneme[-1].isdigit():
                        num_sylls += 1

    ❽ return num_sylls

```

Listing 8-5: Imports modules, loads dictionaries, and counts syllables

After some familiar imports, load the *missing_words.json* file that contains all the words and syllable counts missing from CMUdict. Using `json.load()` restores the dictionary that was stored as a string. Next, turn the CMUdict corpus into a dictionary using the `dict()` method ❶.

Define a function called `count_syllables()` to count syllables. It should take both words *and* phrases, because you'll ultimately want to pass it lines from a haiku. Prep the words as you did previously in the *missing_words_finder.py* program ❷.

Assign a `num_sylls` variable to hold the syllable count and set it to 0 ❸. Now start looping through the input words, stripping punctuation and 's from the ends. Note that you can get tripped up by the format of the apostrophe, so two versions are supplied: one with a straight apostrophe and one with a curly apostrophe ❹. Next, check whether the word is a member of the small dictionary of missing words. If the word is found, add the dictionary value for the word to `num_sylls` ❺. Otherwise, start looking through the phonemes, which represent a value in CMUdict; for each phoneme, look through the strings that make it up ❻. If you find a digit at the end of the string, then you know that phoneme is a vowel. To illustrate using the word *aged*, only the first string (highlighted in gray here) ends with a digit, so the word contains one vowel:

```
[['EY1', 'JH', 'D'], ['EY1', 'JH', 'IH0', 'D']]
```

Note that you use the first value ([0]) in case there are multiple pronunciations; remember that CMUdict represents each pronunciation in a nested list. This may result in the occasional error, as the proper choice will depend on context.

Check whether the end of the phoneme has a digit, and if it does, add 1 to `num_sylls` ❼. Finally, return the total syllable count for the word or phrase ❽.

Defining the main() Function

Completing the program, **Listing 8-6** defines and runs the `main()` function. The program will call this function when the program is run in stand-alone mode—for example, to spot-check a word or phrase—but it won't be called if you import `syllable_counter` as a module.

```

def main():
    ❶ while True:
        print("Syllable Counter")
        ❷ word = input("Enter word or phrase; else press Enter to Exit: ")
        ❸ if word == '':
            sys.exit()
        ❹ try:
            num_syllables = count_syllables(word)
            print("number of syllables in {} is: {}".format(word, num_syllables))
            print()
        except KeyError:
            print("Word not found. Try again.\n", file=sys.stderr)
    ❺ if __name__ == '__main__':
        main()

```

Listing 8-6: Defines and calls the `main()` function

Define the `main()` function and then start a `while` loop ❶. Ask the user to input a word or phrase ❷. If the user presses ENTER with no input, the program exits ❸. Otherwise, start a `try-except` block so the program won't crash if a user enters a word not found in either dictionary ❹. An exception should be raised only in stand-alone mode, as you have already prepared the program to run on the haiku-training corpus with no exceptions. Within this block, the `count_syllables()` function is called and passed the input, and then the results are displayed in the interactive shell. End with the standard code that lets the program run stand-alone or as a module in another program ❺.

A Program to Check Your Program

You have carefully tailored the syllable-counting program to ensure it will work with the training corpus. As you continue with the haiku program, you may want to add a poem or two to this corpus, but adding new haiku might introduce a new word that isn't in either the CMUdict or your exceptions dictionary. Before you go back and rebuild the exceptions dictionary, check whether you really need to do so.

Listing 8-7 will automatically count the syllables in each word in your training corpus and display any word(s) on which it failed. You can download this program from <https://www.nostarch.com/impracticalpython/> as `test_count_syllables_w_full_corpus.py`. Keep it in the same folder as `count_syllables.py`, `train.txt`, and `missing_words.json`.

test_count_syllables_w_full_corpus.py

```
import sys
import count_syllables

with open('train.txt.') as in_file:
    ❶ words = set(in_file.read().split())

    ❷ missing = []

    ❸ for word in words:
        try:
            num_syllables = count_syllables.count_syllables(word)
            ##print(word, num_syllables, end='\n') # uncomment to see word counts
        ❹ except KeyError:
            missing.append(word)

    ❺ print("Missing words:", missing, file=sys.stderr)
```

Listing 8-7: Attempts to count syllables in words in a training corpus and lists all failures

Open your updated `train.txt` training corpus and load it as a set to remove duplicates ❶. Start an empty list, called `missing`, to hold any new words for which syllables can't be counted ❷. Words in `missing` won't be in CMUdict or in your `missing_words` dictionary.

Loop through the words in the new training corpus ❸ and use a try-except block to handle the `KeyError` that will be raised if `count_syllables.py` can't find the word ❹. Append this word to the `missing` list and then display the list ❺.

If the program displays an empty list, then all the words in the new haiku are already present in either CMUdict or *missing_words.json*, so you don't need to make any adjustments. Otherwise, you have the choice of manually adding the words to the *missing_words.json* file or rerunning *missing_words_finder.py* to rebuild *missing_words.json*.

Summary

In this chapter, you've learned how to download NLTK and use one of its datasets, the Carnegie Mellon Pronouncing Dictionary (CMUdict). You checked the CMUdict dataset against a training corpus of haiku and built a supporting Python dictionary for any missing words. You saved this Python dictionary as persistent data using JavaScript Object Notation (JSON). Finally, you wrote a program that can count syllables. In **Chapter 9**, you'll use your syllable-counting program to help you generate novel haiku.

Further Reading

Virtual Muse: Experiments in Computer Poetry (Wesleyan University Press, 1996) by Charles O. Hartman is an engaging look at the early collaboration between humans and computers to write poetry.

Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit (O'Reilly, 2009) by Steven Bird, Ewan Klein, and Edward Loper is an accessible introduction to NLP using Python, with lots of exercises and useful integration with the NLTK website. A new version of the book, updated for Python 3 and NLTK 3, is available online at <http://www.nltk.org/book/>.

"The Growing Importance of Natural Language Processing" by Stephen F. DeAngelis is a *Wired* magazine article on the expanding role of NLP in big data. An online version is available at <https://www.wired.com/insights/2014/02/growing-importance-natural-language-processing/>.

Practice Project: Syllable Counter vs. Dictionary File

Write a Python program that lets you test *count_syllables.py* (or any other syllable-counting Python code) against a dictionary file. After allowing the user to specify how many words to check, choose the words at random and display a listing of each word and its syllable count on separate lines. The output should look similar to this printout:

```
ululation 4
intimated 4
sand 1
worms 1
leatherneck 3
contenting 3
scandals 2
livelihoods 3
intertwining 4
beaming 2
untruthful 3
advice 2
accompanying 5
deathly 2
hallos 2
```

Downloadable dictionary files are listed in **Table 2-1** on **page 20**. You can find a solution in the appendix that can be downloaded from <https://www.nostarch.com/impracticalpython/> as *test_count_syllables_w_dict.py*.